1 Modern Symmetric-Key Ciphers

The classical ciphers of the earlier sections work on letters of a 26-symbol alphabet. A computer, however, has to protect text, numbers, images, audio and video alike, and all of these are stored as bits. Modern ciphers are therefore **bit-oriented**: the plaintext, the ciphertext and the key are bit strings, and the operations are bit operations (permutation of bit positions, table lookup on small groups of bits, XOR). They come in two families, exactly as before: **block ciphers**, which encrypt a fixed-size block of n bits at a time, and **stream ciphers**, which encrypt a stream r bits at a time.

1.1 Modern block ciphers and padding

A **modern block cipher** encrypts an n-bit plaintext block with a k-bit key into an n-bit ciphertext block, and decryption must be the exact inverse, so the encryption mapping has to be a one-to-one function on the 2^n possible blocks. Since the cipher only accepts whole blocks, a message whose length is not a multiple of n is first **padded**. If $|M|$ is the message length and $|Pad|$ the padding length, we need $|M| + |Pad| \equiv 0 \pmod{n}$, which gives directly

$$|Pad| = (-|M|) \bmod n.$$

Only the last block carries padding; every other block is a full block of message bits.

WORKED EXAMPLE : padding (slide example)

Q. How many padding bits must be added to a message of 100 characters if 8-bit ASCII is used and the block cipher accepts 64-bit blocks?
A. The message is $100 \times 8 = 800$ bits. Then $|Pad| = (-800) \bmod 64$. Since $800 = 12 \times 64 + 32$, we get $-800 \equiv -32 \equiv 32 \pmod{64}$, so **32 bits** of padding (e.g. 0s) are added. The padded plaintext is 832 bits = **13** blocks of 64 bits, and the cipher is applied 13 times; only the 13th block contains padding.
Check: $832/64 = 13$ exactly.
Q (variant). 75 characters of 8-bit ASCII, 128-bit blocks (AES)? **A.** $|M| = 600$, $|Pad| = (-600) \bmod 128 = 640 - 600 =$ **40** bits, giving 640 bits = **5 blocks**.

Substitution or transposition at bit level. A block cipher can in principle be designed either as a transposition (rearrange the n bits) or as a substitution (replace the whole n-bit block by another). A transposition only moves bits, so it *preserves the number of 1s*: if the ciphertext has ten 1s, so does the plaintext. Eve then only has to try the $\binom{64}{10} = 151\,473\,214\,816 \approx 2^{37}$ blocks with ten 1s instead of all 2^{64} blocks. A substitution has no such invariant, which is why a modern block cipher is designed as a substitution cipher; transposition units appear only as components inside it.

Full-size key ciphers and why they are impractical. A **full-size key** cipher allows every possible mapping as a key. A full-size key transposition cipher on n bits has $n!$ keys, and a full-size key substitution cipher has $(2^n)!$ keys (every permutation of the 2^n blocks). For $n = 64$ the key would need $\lceil \log_2 64! \rceil = 296$ bits for transposition and $\log_2(2^{64})! \approx 2^{70}$ bits for substitution, i.e. a key larger than any storage. Real ciphers are therefore **partial-size key** ciphers: DES uses a 56-bit key, so it realises only 2^{56} of the $(2^{64})!$ possible substitutions. The design task is to make those 2^{56} mappings look like randomly chosen ones, which is done by combining small components.

1.2 Components of a modern block cipher

A modern block cipher is not built as one huge substitution table. It is a combination of **transposition units (P-boxes)**, **substitution units (S-boxes)** and a few simple operations (XOR, circular shift, swap, split and combine), each small enough to be implemented and analysed.

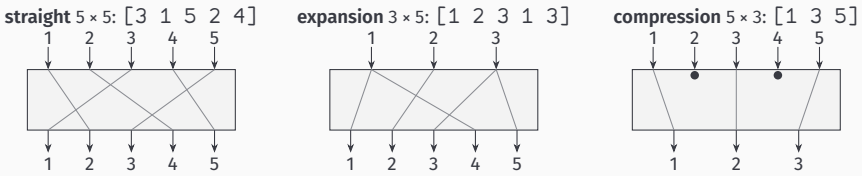
P-boxes ★

▸ **Likely this year:** "Differentiate straight, expansion and compression P-boxes; which are invertible and why?" or "Design an 8 × 8 permutation table that …".

A **P-box** (permutation box) is the bit-level analogue of a transposition cipher: it moves bits to new positions. It is described by a **permutation table** with one entry per *output* position; entry j says which input bit is copied to output j. Three types occur.

- A **straight P-box** has n inputs and n outputs, and its table is a permutation of $1..n$. There are $n!$ straight P-boxes (all 6 for $n = 3$). The DES initial permutation is a 64 × 64 straight P-box (table starting `58 50 42 34 …`).
- An **expansion P-box** has n inputs and $m > n$ outputs. Its table has m entries, of which $m - n$ are repeats, because some inputs are wired to two outputs. The slides' 12 × 16 example `01 09 10 11 12 01 02 03 03 04 05 06 07 08 09 12` repeats inputs 1, 3, 9 and 12.
- A **compression P-box** has n inputs and $m < n$ outputs. Its table has m entries drawn from $1..n$ with $n - m$ values missing: those inputs are **blocked** and never reach the output. The slides' 32 × 24 example blocks inputs 7, 8,

9, 15, 16, 23, 24 and 25.



Invertibility. A straight P-box is invertible: since every input appears exactly once in the table, each output position can be sent back to where it came from. The inverse table is found by writing, for each input position i, the output position that holds it. A compression P-box is *not* invertible, because a blocked input bit is simply lost and the inverse would have to invent its value. An expansion P-box is *not* invertible either: an input copied to two outputs would have two candidate sources in the reverse direction, and if the received bits disagree there is no definite answer (more precisely, its reverse is a compression, not an inverse). Only straight P-boxes are invertible. This is exactly why AES (all components invertible) uses only straight permutations, while DES (Feistel) can afford an expansion P-box inside its round function.

WORKED EXAMPLE : designing and inverting a straight P-box (slide example)

Q. Design an 8 × 8 permutation table for a straight P-box that moves the two middle bits (bits 4 and 5) of the input to the two ends (bits 1 and 8) of the output, without changing the relative positions of the other bits. Give the inverse table and apply the P-box to `10011100`.
A. Output bit 1 must take input bit 4 and output bit 8 must take input bit 5. The remaining outputs 2–7 take inputs 1, 2, 3, 6, 7, 8 in their original order. Hence the table is

`[4 1 2 3 6 7 8 5]`.

Inverse: input 1 went to output 2, input 2 to 3, input 3 to 4, input 4 to 1, input 5 to 8, input 6 to 5, input 7 to 6, input 8 to 7, so the inverse table is `[2 3 4 1 8 5 6 7]`.
Apply: input `10011100` has bits $b_1..b_8 = 1,0,0,1,1,1,0,0$. Output = $b_4 b_1 b_2 b_3 b_6 b_7 b_8 b_5$ = **11001001**.
Check: applying the inverse table to `11001001` returns `10011100`; the number of 1s (four) is unchanged, as it must be for any P-box.

S-boxes

An **S-box** (substitution box) is a miniature substitution cipher on a few bits: it takes an n-bit input and produces an m-bit output, where m and n need not be equal. It is normally given as a **lookup table**: some input bits select the row, the rest select the column, and the cell holds the output. In DES, for instance, a 6-bit input chooses one of 4 rows by its two outer bits and one of 16 columns by its four middle bits, and the cell is a 4-bit value.

Linear versus nonlinear. Write the inputs as $x_1..x_n$ and the outputs as $y_1..y_m$. The S-box is **linear** if every output is an XOR of inputs, $y_i = a_{i1}x_1 \oplus a_{i2}x_2 \oplus \cdots \oplus a_{in}x_n$ with constants $a_{ij} \in \{0,1\}$; then the whole box is a matrix product over GF(2). For example, $y_1 = x_1 \oplus x_2 \oplus x_3$, $y_2 = x_1$ is linear, with matrix $\left(\begin{smallmatrix}1&1&1\\1&0&0\end{smallmatrix}\right)$. If any output contains a product of inputs, such as $y_1 = x_1x_3 \oplus x_2$, the box is **nonlinear**. A cipher built only from linear boxes, XORs and P-boxes would be linear as a whole, so every ciphertext bit would be a fixed XOR of plaintext and key bits, and a handful of known plaintext–ciphertext pairs would give a solvable linear system for the key. The S-boxes are the only nonlinear part of a block cipher, and they must be nonlinear.

Invertibility. An S-box can be inverted only if it is a one-to-one map, which requires $m = n$ (a $6 \to 4$ box maps 64 inputs onto 16 outputs, so each output has 4 preimages). The slides' 3 × 3 example uses the leftmost bit as the row and the two rightmost bits as the column:

enc	00	01	10	11
0	011	101	111	100
1	000	010	001	110

dec	00	01	10	11
0	100	110	101	000
1	011	001	111	010

Input `001` (row 0, column 01) encrypts to `101`, and `101` (row 1, column 01) in the decryption table gives back `001`; a script confirms that the two tables undo each other for all 8 inputs.

XOR, circular shift, swap, split and combine

Exclusive-or is addition in GF(2^n), and five properties make it the standard way to mix in a key: **closure** (the XOR of two n-bit words is an n-bit word), **associativity**, **commutativity**, an **identity** 00…0, and an **inverse** for every

word, namely the word itself, since $x \oplus x = 00\ldots0$. The last property is the important one: whatever is XORed in can be removed by XORing the same value again, $(x \oplus k) \oplus k = x$. A key mixed in this way is called a **whitener**.

A **circular shift** rotates an n-bit word left or right by k positions. It is invertible, since a left shift by k is undone by a right shift by k, and shifting by k is the same as shifting by k mod n. A **swap** is the special circular shift by $k = n/2$, exchanging the left and right halves; doing it twice restores the word, so it is **self-invertible**. A **split** cuts an n-bit word into two $n/2$-bit halves and a **combine** concatenates two halves; each is the inverse of the other.

1.3 Product ciphers: confusion, diffusion and rounds ★

▸ **Likely this year:** One-line theory: "Define confusion and diffusion; which component provides which?" or "What is the avalanche effect?"

A single small component is easy to analyse and therefore easy to break; a full-size substitution table is secure but impossible to store. Shannon's answer was the **product cipher**: a cipher that alternates substitutions, permutations and key mixing, so that simple and cheap parts combine into a mapping that behaves like a random one. He named the two properties such a cipher must have.

Diffusion hides the relationship between the ciphertext and the plaintext: each ciphertext bit should depend on many or all plaintext bits, so that changing one plaintext bit changes many ciphertext bits. This frustrates an attacker who uses ciphertext statistics (letter or bit frequencies) to guess the plaintext, because any local pattern of the plaintext is spread over the whole block.

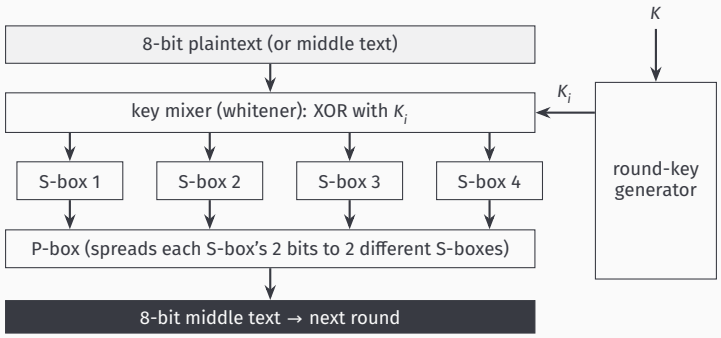
Confusion hides the relationship between the ciphertext and the key: each ciphertext bit should depend on the key in a complicated way, so that changing one key bit changes most ciphertext bits. This frustrates an attacker who uses known ciphertext to solve for the key.

Why S-boxes give confusion. The key enters the cipher only through XOR, and XOR and P-boxes are linear over GF(2). If the cipher had no S-boxes, every ciphertext bit would be a fixed XOR of plaintext bits and key bits, and a few plaintext–ciphertext pairs would give a linear system whose solution is the key. The nonlinear S-box placed after the key mixing breaks this: its output is a nonlinear function of (data $\oplus$ key), so after a few rounds the ciphertext is a high-degree nonlinear function of the key that cannot be solved for. That is precisely confusion.

Why P-boxes give diffusion. Suppose one input bit of a round changes. It enters one S-box, whose output may change in all its bits (say 2 bits for a 2×2 box). The P-box then sends those changed bits to *different* S-boxes of the next round, where each again changes up to 2 bits, so the change can grow from 1 bit to 2, 4, 8, ... bits in successive rounds. Without the P-box the changed bits would stay inside the same S-box forever and the change would never spread; the permutation between rounds is what carries it across the block. That is diffusion.

Avalanche. The observable result of good confusion and diffusion is the **avalanche effect**: flipping one bit of the plaintext or of the key flips about half of the ciphertext bits. Section **??** shows DES going from 1 differing bit to 29 differing bits out of 64.

Rounds. One layer of S-boxes and P-boxes spreads a bit change only a little, so the layer is repeated. An **iterated product cipher** applies the same **round** N times; each round consists of a **key mixer** (whitener, XOR with a round key), a layer of S-boxes and a P-box. The text between two rounds is called the **middle text**, and a **key schedule** (round-key generator) derives the round keys $K_1, \ldots, K_N$ from the cipher key, so that each round mixes in a different key. Decryption runs N inverse rounds with the round keys in reverse order.



One round of the slides' 8-bit product cipher (the slide shows two such rounds).

1.4 Feistel ciphers ★

★ **In the papers:** 2024–25 Q6 (5 marks) opens with "what Feistel structure is prevalent in DES": the derivation below is that part of the answer.

Modern block ciphers fall into two classes. **Feistel ciphers** (DES) use invertible, self-invertible *and noninvertible* components; **non-Feistel ciphers** (AES) use only invertible components. The problem the Feistel design solves is this: noninvertible components such as an expansion P-box or a $6 \to 4$ S-box mix bits very effectively, but a cipher must be decryptable, and a noninvertible component cannot be run backwards. Feistel's idea is to collect

all the noninvertible parts into one function f and never invert it. Its output is only ever *XORed* onto the data, and since XOR is self-inverse, the receiver can cancel it by computing the same f value again and XORing once more.

Step 1: the idea in its simplest form. Let f depend only on the key. Then

$$\text{encryption: } C = P \oplus f(K), \qquad \text{decryption: } C \oplus f(K) = P \oplus f(K) \oplus f(K) = P \oplus 00\ldots0 = P.$$

Nothing here requires f^{-1}; decryption simply computes $f(K)$ forwards.

WORKED EXAMPLE : the slides' first Feistel example

Q. Plaintext and ciphertext are 4 bits, the key is 3 bits. f takes the first and third bits of the key, reads them as a decimal number, squares it, and writes the result as 4 bits. Encrypt and decrypt $P = 0111$ with $K = 101$.
A. First and third key bits: $1, 1 \to (11)_2 = 3$; $3^2 = 9 = 1001$, so $f(101) = 1001$.
Encryption: $C = P \oplus f(K) = 0111 \oplus 1001 =$ **1110**.
Decryption: $C \oplus f(K) = 1110 \oplus 1001 = 0111 = P$ ✓.
f is noninvertible (e.g. keys 101 and 111 both give 1001, since bit 2 is ignored), yet decryption works because only f, never f^{-1}, is needed.

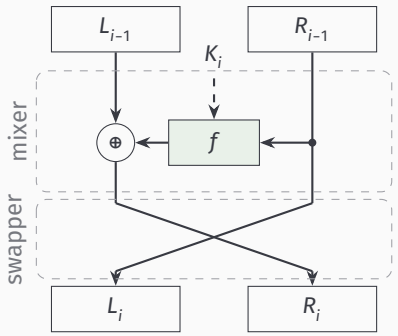
Step 2: let the data enter f. A function of the key alone adds nothing per block (it is a fixed XOR, i.e. a one-key Vernam). To make f depend on the data while keeping decryption possible, the block is **split** into a left half L and a right half R; R feeds f and L is XORed with the output. This unit is the **mixer**:

$$L_2 = L_1 \oplus f(R_1, K), \qquad R_2 = R_1.$$

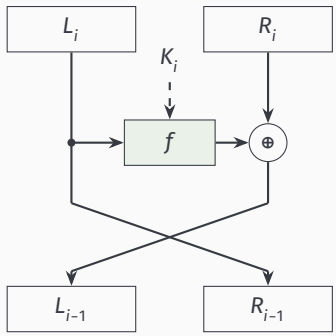
The receiver gets $R_2 = R_1$ unchanged, so it can recompute exactly the same $f(R_1, K)$ and cancel it: $L_2 \oplus f(R_2, K) = L_1 \oplus f(R_1, K) \oplus f(R_1, K) = L_1$. The mixer is therefore **self-invertible**: applying it twice with the same key is the identity.

Step 3: the swapper and more rounds. The mixer has one flaw: R is never changed, so half of the plaintext appears in clear in the ciphertext. The fix is to add a **swapper** after the mixer, so that the next round modifies the other half, and to repeat for several rounds. One Feistel round i is then

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i).$$



encryption round:
$L_i = R_{i-1}$, $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$



inverse round (same K_i), f only forwards:
$R_{i-1} = L_i$, $L_{i-1} = R_i \oplus f(L_i, K_i)$

★ MUST KNOW : why a Feistel round is invertible even if f is not

Given the round output (L_i, R_i), the first equation hands back $R_{i-1} = L_i$ directly. Knowing R_{i-1}, the receiver can *recompute* $f(R_{i-1}, K_i)$ forwards and XOR it away:

$$R_i \oplus f(L_i, K_i) = L_{i-1} \oplus f(R_{i-1}, K_i) \oplus f(R_{i-1}, K_i) = L_{i-1}.$$

f is only ever evaluated, never inverted, so it may be many-to-one (expansion P-box, $6 \to 4$ S-boxes). Since the mixer and the swapper are each self-inverse, decryption is the same structure run with the round keys in reverse order $(K_N, \ldots, K_1)$.

In the slides' two-round final design (slide L7-29), both rounds end with a swapper, so the decryption circuit applies the swapper first and then the mixer, round keys K_2 then K_1. DES removes the swapper from the *last* round; then encryption and decryption are literally the same algorithm and only the key order changes. The toy below uses that DES convention.

WORKED EXAMPLE : two-round Feistel cipher with a noninvertible f

Q. An 8-bit Feistel cipher has 4-bit halves and two rounds; round 1 has a swapper, round 2 does not (DES convention). The round function is $f(R, K) = (R \oplus K)^2 \bmod 16$. Encrypt $L_0R_0 = 1010\ 0111$ with $K_1 = 0100$, $K_2 = 1001$, then decrypt with the same algorithm and the keys reversed. Show that f is not invertible.
A. f is not invertible: $0011^2 = 9$ and $0101^2 = 25 \equiv 9 \pmod{16}$, so two inputs give 1001; in fact $x^2 \bmod 16$ only takes the values 0, 1, 4, 9.

Round	input L, R	$R \oplus K$	f	$L \oplus f$	output (after swap?)
Enc 1 ($K_1 = 0100$)	1010, 0111	0011	1001	0011	swap → 0111, 0011
Enc 2 ($K_2 = 1001$)	0111, 0011	1010	0100	0011	no swap → **0011, 0011**
Dec 1 ($K_2 = 1001$)	0011, 0011	1010	0100	0111	swap → 0011, 0111
Dec 2 ($K_1 = 0100$)	0011, 0111	0011	1001	1010	no swap → **1010, 0111**

Ciphertext = **0011 0011**; decryption returns 1010 0111 = P ✓. Each decryption round recomputes exactly the f value of the matching encryption round (1001 and 0100), because the half fed to f is passed through unchanged.

1.5 Non-Feistel ciphers

A **non-Feistel cipher** uses only invertible components, and every component of the encryption has a matching inverse component in the decryption. This constrains the parts: S-boxes must have as many output bits as input bits, and expansion and compression P-boxes are not allowed, because they cannot be inverted. Since there is no f whose output is XORed in, there is also no need to split the block into halves; each round transforms the whole block. The slides' 8-bit product cipher above is a non-Feistel cipher: XOR (self-invertible), 2×2 invertible S-boxes and a straight P-box, so each round is invertible and decryption runs the inverse rounds with keys K_2, K_1. AES is the real example.

	Feistel (DES)	**Non-Feistel** (AES)
Components	invertible, self-invertible and noninvertible	only invertible ($n \times n$ S-boxes, straight P-boxes)
Block handling	split into L, R; mixer $L \oplus f(R, K)$, then swapper	whole block in every round
Decryption	same structure, round keys reversed; f never inverted	inverse components in reverse order, round keys reversed

1.6 Attacks on modern block ciphers

Brute force on the key is normally infeasible because keys are long (but 2^{56} for DES is not). The serious attacks exploit the structure of the rounds.

- **Differential cryptanalysis** (Biham and Shamir) is a **chosen-plaintext** attack. Eve submits pairs of plaintexts with a chosen difference $P_1 \oplus P_2$ and studies the output difference $C_1 \oplus C_2$; a weak cipher propagates certain differences with high probability, which reveals key bits. The slides' extreme case is a cipher that is only an XOR with the key: $C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2$, so the difference passes through untouched and is independent of the key. Nonlinear S-boxes are what destroy this predictability.
- **Linear cryptanalysis** (Matsui) is a **known-plaintext** attack. If an S-box is not fully nonlinear, some XOR of input and output bits equals an XOR of key bits with probability noticeably different from $1/2$; collecting many plaintext–ciphertext pairs and counting how often such an equation holds reveals those key bits.

1.7 Modern stream ciphers

A **modern stream cipher** encrypts and decrypts r bits at a time: the plaintext $P = p_n \ldots p_2 p_1$, ciphertext $C = c_n \ldots c_2 c_1$ and key stream $K = k_n \ldots k_2 k_1$ consist of r-bit words, and $c_i = p_i \oplus k_i$, $p_i = c_i \oplus k_i$. Encryption and decryption are a single XOR, so the cipher is fast and needs no padding; all of its security lies in the **key stream**, which must look random and must never be reused. Stream ciphers are classified by how the key stream is produced.

Synchronous stream ciphers. In a **synchronous** stream cipher the key stream is *independent* of the plaintext and the ciphertext; it is generated from the key alone, and sender and receiver must stay in step (synchronised) with each other. The ideal case is the **one-time pad**: a truly random key stream as long as the message, used once, giving perfect secrecy, at the price that a key as long as all the traffic must be shared in advance. Practical synchronous ciphers replace the random pad by a pseudorandom generator such as a **feedback shift register** (FSR). An FSR has m one-bit cells $b_{m-1} \ldots b_1 b_0$; at each step b_0 is output as a key bit, all cells shift right, and the new leftmost bit is computed by a **feedback function** of the old cells. In a **linear** FSR (LFSR) the feedback is an XOR of selected cells, so it costs one gate. With a well-chosen feedback an m-cell LFSR runs through all $2^m - 1$ nonzero states before repeating, the maximum period. Because it is linear, however, about $2m$ consecutive key-stream bits are enough to solve for the feedback taps and predict the rest, so real designs use a **nonlinear** FSR or combine several LFSRs nonlinearly.

WORKED EXAMPLE : LFSR key stream (Forouzan style)

Q. Create a 4-cell LFSR with feedback $b_4 = b_1 \oplus b_0$ (the new b_3). Show the output for 20 shifts if the seed $b_3 b_2 b_1 b_0$ is 0001, and find the period.
A. Each step outputs b_0, then the state becomes $(b_1 \oplus b_0)\, b_3\, b_2\, b_1$.

step	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
state	0001	1000	0100	0010	1001	1100	0110	1011	0101	1010	1101	1110	1111	0111	0011	**0001**
out b_0	1	0	0	0	1	0	0	1	1	0	1	0	1	1	1	1

At step 15 the state is the seed again, so the key stream repeats with period **15** = $2^4 - 1$ (maximum: every nonzero state appears once). The 20 output bits are **1000 1001 1010 1111 0001**.
Check: the bits 16–20 (1 0001) repeat bits 1–5 (1 0001), as the period predicts.

Nonsynchronous (self-synchronising) stream ciphers. In a **nonsynchronous** stream cipher each key-stream element depends on previous plaintext or ciphertext elements. The usual form is **self-synchronising**: the key stream is computed from the key and the last few ciphertext bits (as in CFB mode). If a ciphertext bit is corrupted or lost, the receiver's key stream is wrong only while the bad bit is inside its window, and decryption recovers by itself once the window has passed.

Error behaviour. In a synchronous cipher a flipped ciphertext bit spoils only the matching plaintext bit, but a lost or inserted bit shifts the two key streams against each other and garbles everything after it until the parties resynchronise. In a self-synchronising cipher a single error spoils a few following bits as well, but a lost bit is recovered from automatically.

Corrections to the notes / slides

- Feistel example (L7 slide 25): the problem says f uses the *first and third* key bits, but the solution "extracts the first and second bits to get 11". For $K = 101$ the first and second bits are $10 = 2$, giving $f = 0100$; only the first and third bits give $11 = 3$ and the stated $f = 1001$, $C = 1110$. Use first and third.
- Stream ciphers (L7 slide 39): the line reads "In a synchronous stream cipher the key is independent of the plaintext or ciphertext"; it is easily read as "asynchronous" (the old version of these notes did so). The key-independent class is **synchronous**. Some books (Stallings) call the *other* class, the self-synchronising ciphers, "asynchronous", so do not mix the words up.
- Nonlinear S-box (L7 slide 18): over GF(2), $x^3 = x$ and $x^2 = x$, so the slide's $y_1 = x_1^3 + x_2 = x_1 + x_2$ is actually linear; the box is nonlinear because of the product term $x_1 x_2$ in $y_2 = x_1^2 + x_1 x_2 + x_3$.

Check your understanding

1. **Why must a modern block cipher be a substitution and not a transposition cipher?** A transposition preserves the number of 1s in the block, so from a ciphertext with w ones Eve only has to try the $\binom{n}{w}$ blocks of weight w; a substitution has no such invariant and leaves all 2^n candidates.
2. **How many padding bits for 1000 bits of message with 64-bit blocks?** $(-1000) \bmod 64 = 1024 - 1000 = 24$ bits, giving 16 blocks.
3. **Why is a full-size key substitution cipher impractical for $n = 64$?** It needs $(2^{64})!$ keys, i.e. a key of about 2^{70} bits; real ciphers use partial-size keys (DES: 56 bits) and build a random-looking mapping from small components.
4. **Differentiate the three P-boxes and say which are invertible.** Straight $n \to n$ is a permutation and is invertible; expansion $n \to m$ ($m > n$) repeats inputs and compression $n \to m$ ($m < n$) blocks inputs, and neither is invertible, because a blocked bit is lost and a repeated bit has no unique source.
5. **Which component gives confusion and which gives diffusion, and why?** S-boxes give confusion, because they are the only nonlinear part and make the ciphertext a nonlinear function of the key; P-boxes give diffusion, because they carry the changed output bits of one S-box to several S-boxes of the next round so that a change spreads over the block.
6. **Why can a Feistel cipher use a noninvertible round function?** Because f's output is only XORed onto the other half and f's input R_{i-1} is passed on unchanged as L_i; the receiver recomputes $f(L_i, K_i)$ forwards and XORs it away, so f^{-1} is never needed.
7. **What happens if a Feistel cipher has no swapper?** The right half is never modified, so half of the plaintext appears unchanged in the ciphertext.
8. **Can an S-box with 6 inputs and 4 outputs be invertible?** No: 64 inputs map onto 16 outputs, so each output has 4 preimages; only $n \times n$ S-boxes can be invertible.
9. **Differentiate synchronous and nonsynchronous stream ciphers.** In a synchronous cipher the key stream depends only on the key (OTP, LFSR), so an inserted or lost bit desynchronises the receiver; in a nonsynchronous cipher it also depends on earlier plaintext or ciphertext, and the self-synchronising form recovers automatically after an error.

10. **What is the maximum period of a 5-cell LFSR?** $2^5 - 1 = 31$, since the all-zero state is excluded (it would stay zero forever).